



## CI/CD for Agent Deployments

Google Cloud

Proprietary & Confidential

1

Welcome to Module 2: CI/CD for Agent Deployments.

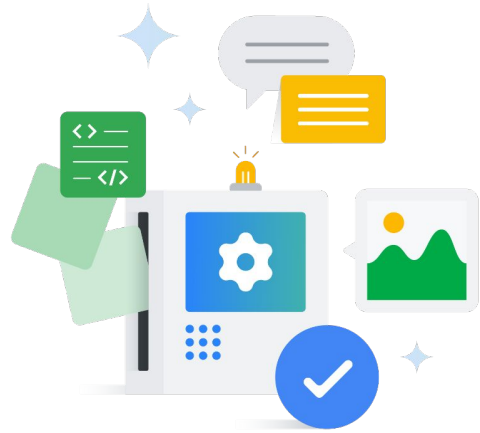
In the previous module, we introduced the concept of AgentOps and discussed the operational challenges of running agents in production.

In this module, we focus specifically on how to operationalize agent deployments using CI/CD pipelines.

CI/CD pipelines allow teams to automate the process of building, testing, and deploying agents, which is critical when agents evolve frequently due to changes in prompts, models, tools, or application logic.

## In this module, you learn to ...

- 01 Leverage CI/CD tooling and patterns for agentic solutions on Google Cloud
- 02 Select the deployment target for agents on Google Cloud
- 03 Build a complete CI/CD pipeline for an agent



In this module, we'll cover three main learning goals.

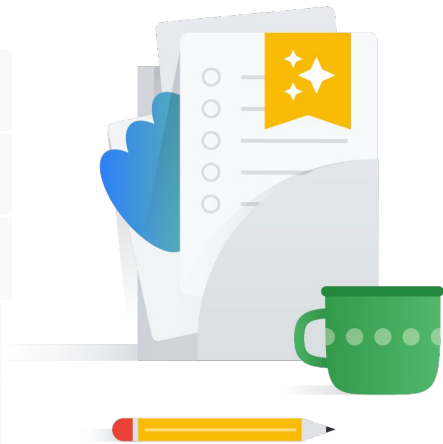
First, we'll review how CI/CD principles apply to agentic systems.

Second, we'll look at deployment targets for agents on Google Cloud, including the tradeoffs between different runtime environments.

Finally, we'll walk through how to build a full CI/CD pipeline for an agent, including build automation, artifact management, and deployment.

# Topics

- |    |                                            |
|----|--------------------------------------------|
| 01 | CI/CD review                               |
| 02 | Agentic deployment targets                 |
| 03 | CI/CD tooling and patterns on Google Cloud |
| 04 | Cloud Build automation                     |
| 05 | Lab: CI/CD for Agents on Google Cloud      |



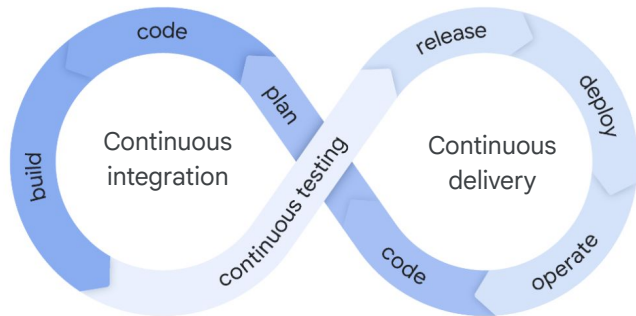
This module is organized into five sections.

1. We'll begin with a quick CI/CD review
2. Followed by a discussion of agent deployment targets on Google Cloud
3. Then we'll examine CI/CD tooling and patterns, including artifact management and infrastructure as code
4. Next, we'll look at Cloud Build automation and how pipelines are defined
5. Finally, you'll complete a hands-on lab where you build and deploy an agent using a CI/CD pipeline

# CI/CD:

## Continuous integration and continuous delivery

**Software development practice** that automates the process of building, testing, and deploying code changes.



CI/CD stands for **Continuous Integration and Continuous Delivery**.

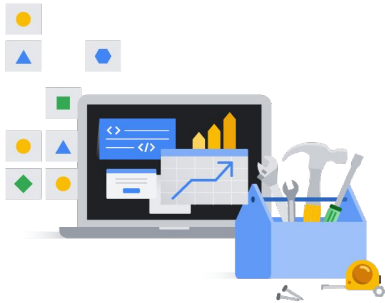
It's a software development practice designed to automate the process of integrating code changes, running tests, and deploying updates.

Continuous integration focuses on **automatically building and testing code whenever changes are committed**.

Continuous delivery focuses on **releasing validated code into production environments reliably and frequently**.

Together, these practices allow teams to ship changes safely and quickly.

# Continuous integration



The practice of frequently merging code changes into a central repository.

Repository ensures code remains functional through automated:

✓ Builds

✓ Tests

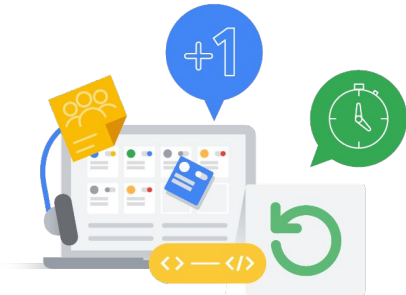
Continuous integration is the practice of frequently merging code changes into a shared repository.

Each change automatically triggers builds and tests to ensure the codebase remains functional.

If a test fails, the issue can be detected immediately before it affects production.

This approach helps maintain a stable codebase and prevents integration problems from accumulating.

# Continuous delivery



Producing software in short cycles and using processes to ensure a reliable release to production.



Helps minimize the risk of loss of service quality.



Might mean pushing releases to production several times a day.

And then there is **continuous delivery**, which is a method where teams produce software in short cycles and use processes that ensure the reliable release of software to production at any time. This helps minimize the risk of loss of service quality. For some companies, this means pushing releases to production several times a day.

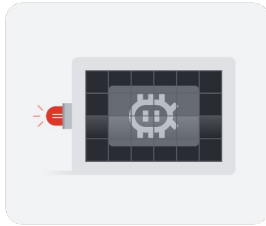
Continuous delivery focuses on **releasing software changes in short, reliable cycles**.

Instead of infrequent large releases, teams deploy smaller updates more frequently.

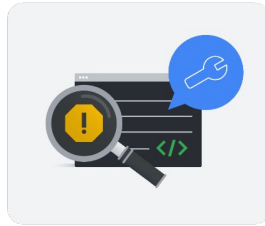
This approach reduces the risk associated with deployments because each change is small and easier to validate.

In some environments, deployments may occur multiple times per day.

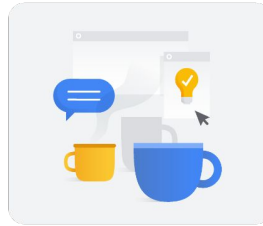
## The benefits of CI/CD



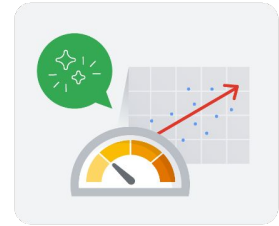
Helps detect bugs sooner.



Helps improve code quality.



Helps enhance collaboration.



Helps reduce the risk of regressions.

Now that you know what CI/CD stands for, let's explore some of the benefits.

For starters, CI/CD can **help detect bugs sooner**, which can prevent them from turning into larger problems later in the development cycle. For example, if all of your users have the same version of the application, developers can test updates on a percentage as small as 1%. Feedback can be used from that 1% to identify and fix any bugs or anomalies in the new update, so that when you roll the update out to the other 99% of users, very few bugs will be visible to them.

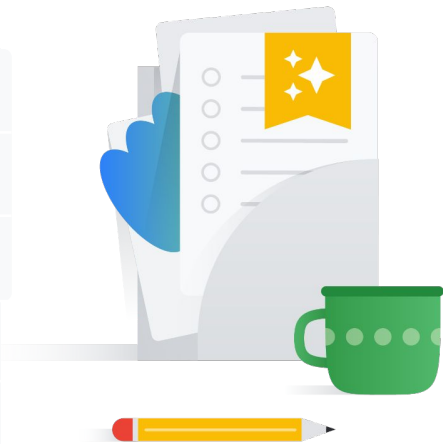
CI/CD can also **help improve code quality**, because frequent integration and testing can encourage developers to write cleaner and more robust code.

It can **help enhance collaboration** since teams work on a shared codebase.

And finally, CI/CD can also **help reduce the risk of regressions**, because merging code frequently minimizes the chances of breaking existing functionalities.

# Topics

- |    |                                            |
|----|--------------------------------------------|
| 01 | CI/CD review                               |
| 02 | Agentic deployment targets                 |
| 03 | CI/CD tooling and patterns on Google Cloud |
| 04 | Cloud Build automation                     |
| 05 | Lab: CI/CD for Agents on Google Cloud      |



Now that we've reviewed CI/CD concepts, the next step is understanding where agents actually run in production.

Google Cloud offers multiple deployment environments for agents, each with different levels of control, scalability, and operational complexity.

# Agentic deployment targets on Google Cloud

| Agent Platform Agent Runtime                                                                                                                                                           | Cloud Run                                                                                                                                                                                               | GKE                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Fully managed runtime purpose-built for AI agents</li><li>• Built-in capabilities like short-term session state and long-term memory</li></ul> | <ul style="list-style-type: none"><li>• Serverless container platform</li><li>• More control over the environment than Agent Engine</li><li>• Cost-effective—can scale down to zero when idle</li></ul> | <ul style="list-style-type: none"><li>• Full Kubernetes cluster running your agent services</li><li>• Provides maximum architectural control</li></ul> |

There are three primary deployment options for agents on Google Cloud.

- First is Agent Platform Agent Runtime, which is a fully managed runtime designed specifically for AI agents.
- Second is Cloud Run, a serverless platform for running containerized services. Cloud Run supports virtually any application you can containerize.
- Third is Google Kubernetes Engine, which provides a fully managed Kubernetes environment with maximum customization.

Each option represents a different balance between simplicity and control.

# Agent Platform Agent Runtime

## The managed solution

|                       |                                                                                                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scalability           | Automatically scales to meet demand and can scale to zero                                                                                                                      |
| Supported languages   | Python, Typescript, Go, Java                                                                                                                                                   |
| Hardware acceleration | Agent orchestration runs in a managed environment (Agent logic runs on CPU); model inference leverages Agent Platform infrastructure (including accelerators where applicable) |
| State management      | Offers native services like Sessions and Memory Bank to handle short-term and long-term conversation context without needing an external database                              |
| Agent sandbox         | Includes a managed code execution environment that allows agents to run AI-generated code securely                                                                             |
| CI/CD integration     | Abstracts away container management; developers typically don't manage Dockerfiles directly                                                                                    |
| Zero infrastructure   | Automatically handles containerization, web servers, and endpoint security                                                                                                     |

Google Cloud

Proprietary & Confidential 10

Gemini Enterprise Agent Platform is the fully managed option.

It automatically handles infrastructure concerns such as containerization, scaling, and endpoint security.

It does not currently support direct GPU attachment for the agent's orchestration code itself. However, you can easily connect your agent to models or services that do run on GPUs.

It also provides built-in features like session management and memory handling.

Because infrastructure management is abstracted away, developers can focus primarily on agent logic.

This option is ideal when you want the fastest path to production with minimal infrastructure overhead.

<https://docs.cloud.google.com/architecture/choose-agentic-ai-architecture-component#agent-runtime-vertex-ai-agent-engine>

# Cloud Run

## The serverless experience

|                              |                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scalability</b>           | Automatically scales to meet demand and can scale to zero                                                                                                           |
| <b>Supported languages</b>   | Runs any standard Docker container image, supports any language or framework that can be containerized                                                              |
| <b>Hardware acceleration</b> | Supports GPUs (e.g., NVIDIA L4) in select regions; scaling behavior and availability differ from standard CPU workloads                                             |
| <b>State management</b>      | To implement persistent memory, connect your service to a managed Google Cloud storage service: Memorystore, Firestore, Cloud SQL, Memory Bank with Cloud Run, etc. |
| <b>Agent sandbox</b>         | Isolate untrusted code with GKE Sandbox                                                                                                                             |
| <b>CI/CD integration</b>     | Plugs easily into existing declarative CI/CD pipelines (like Cloud Build or GitHub Actions)                                                                         |

Cloud Run offers a serverless container runtime.

Developers package their agent as a Docker container and deploy it to Cloud Run.

This approach provides more flexibility than Agent Engine while still maintaining serverless benefits like automatic scaling and scale-to-zero.

Cloud Run also integrates well with existing CI/CD pipelines and supports GPU workloads when needed.

# Google Kubernetes Engine

## The orchestrated experience

|                       |                                                                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scalability           | Custom auto scaling allows for highly customized autoscaling based on metrics                                                                                 |
| Supported languages   | Runs any standard Docker container image, supports any language or framework that can be containerized                                                        |
| Hardware acceleration | Extensive support for GPUs and TPUs                                                                                                                           |
| State management      | To implement persistent memory, connect your service to a managed Google Cloud storage service: Memorystore, Firestore, Cloud SQL, Memory Bank with GKE, etc. |
| Agent sandbox         | Isolate untrusted code with GKE Sandbox                                                                                                                       |
| CI/CD integration     | Plugs easily into existing declarative CI/CD pipelines (like Cloud Build or GitHub Actions)                                                                   |
| More control          | Gives you control over networking, security policies, hardware provisioning, and resource limits                                                              |

Google Cloud

Proprietary & Confidential 12

Google Kubernetes Engine provides the highest level of control.

Teams can manage networking policies, hardware configurations, and advanced scaling behaviors.

GKE is typically used for complex architectures or organizations already operating Kubernetes platforms.

However, it also introduces more operational responsibility compared to managed services like Cloud Run or Agent Engine.

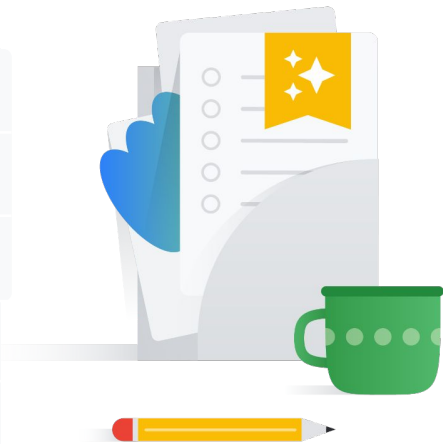
Best For: Long-running autonomous agents, complex multi-agent simulations, and workloads requiring secure code execution.

State Management: Full Stateful Support. Unlike Cloud Run's stateless model, GKE supports StatefulSets and Persistent Volumes (Hyperdisk), allowing agents to maintain "hot" memory locally without constant database round-trips

Pod Snapshots: Instantly save/restore agent memory states to reduce latency.

## Topics

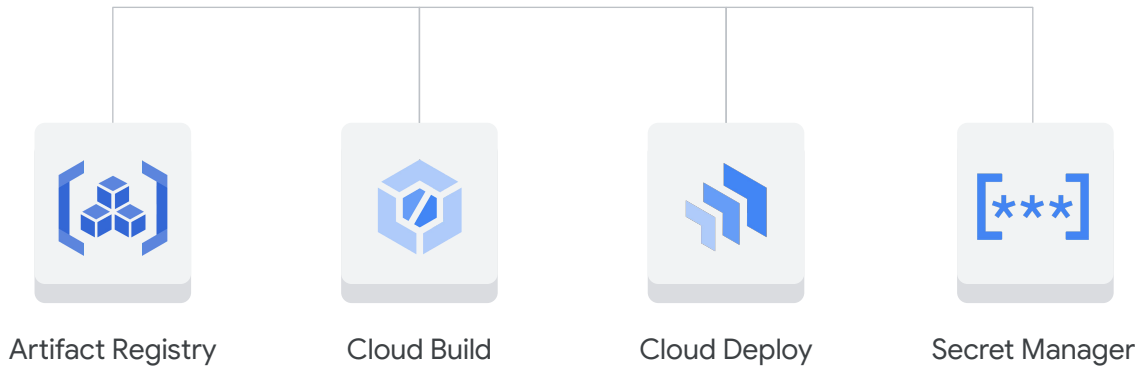
- |    |                                            |
|----|--------------------------------------------|
| 01 | CI/CD review                               |
| 02 | Agentic deployment targets                 |
| 03 | CI/CD tooling and patterns on Google Cloud |
| 04 | Cloud Build automation                     |
| 05 | Lab: CI/CD for Agents on Google Cloud      |



Now that we understand where agents can run, let's examine the tooling used to build and deploy them.

Google Cloud provides several services that together form a complete CI/CD pipeline.

## CI/CD tooling on Google Cloud



The continuous integration and continuous delivery development technique, or CI/CD, helps software development teams ensure that code is of high quality and can be deployed to production reliably and quickly.

Google offers a variety of tools and services to help teams implement CI/CD pipelines, including Artifact Registry, Cloud Build, and Cloud Deploy.

Secret Manager securely stores credentials and sensitive configuration data.

Together, these services enable a fully automated pipeline.

# Artifact Registry



Artifact Registry

- ✓ A fully-managed, secure, and scalable artifact repository
- ✓ Stores, manages, and distributes build artifacts
- ✓ Supports multiple artifact formats, including:
  - Docker images
  - Binaries
  - AAR, APK, JAR, POM, ZIP, etc.
- ✓ Can enable vulnerability scanner to scan images when they are pushed to the registry

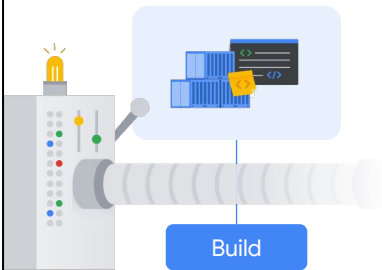
Google's **Artifact Registry** is a fully-managed, secure, and scalable artifact repository for storing, managing, and distributing build artifacts.

Artifact Registry can be used to store artifacts of any type, including source code, binaries, and Docker images.

Artifact Registry is where build artifacts are stored.

- This includes Docker images and other binary artifacts produced during builds.
- It provides secure storage and integrates directly with CI/CD pipelines.
- It also supports vulnerability scanning to detect security issues in container images before deployment.

# Cloud Build



When raw source code is transformed into a deployable package.



Cloud Build



Can link to source code from a variety of repositories



Executes build as a series of build steps in response to specific changes in your code (triggers)



Serverless execution model



Build steps typically include:



Compiling code



Building container images



Running tests



Deploying to test or staging env

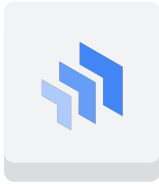
Cloud Build is the core build automation service.

It executes build steps whenever changes are detected in the source repository.

These steps may include compiling code, building container images, running tests, and deploying to environments.

Because Cloud Build is serverless, it scales automatically based on demand.

# Cloud Deploy



Cloud Deploy



Fully managed continuous delivery service for easy scaling



Supports deployment pipelines for GKE and Cloud Run



Connects the build artifacts to the deployment environment (dev, test, staging, prod)



Provides a single place to view and manage all of your deployments

Cloud Deploy manages the continuous delivery portion of the pipeline.

It defines deployment pipelines that move artifacts through environments such as development, staging, and production.

This allows teams to visualize and control the entire deployment process in a single interface.

Cloud Deploy plays a crucial role in the CI/CD pipeline. While CI/CD focuses on automating the entire software delivery process, Cloud Deploy specifically handles the continuous delivery (CD) stage, seamlessly connecting the build artifacts to the deployment environment.

It provides a single place to view and manage all of your deployments, and can be integrated with other Google Cloud services, such as Cloud Monitoring and Cloud Logging.

# Secret Manager



Secret Manager



A fully-managed service to store, manage, and audit access to credentials and secrets



Can store sensitive information of any type, including:

API keys

Database passwords

TLS certificates



Supports versioning and can be integrated with automated rotation workflows



**Never** store secrets in source code

Secret Manager securely stores sensitive data such as API keys, database credentials, and TLS certificates.

Secrets should never be stored directly in source code repositories.

Secret Manager allows controlled access and automatic versioning of secrets.

This improves both security and compliance.

## CI/CD tooling: IaC/ Terraform



Terraform



Terraform is a leading Infrastructure as Code (IaC) tool for provisioning cloud resources



Defines infrastructure in configuration files



Use Terraform to define repeatable, version-controlled agent infrastructure and pipeline resources:

- Cloud Run services

- Service Accounts/IAM policies

- Cloud Build triggers

- Artifact Registry repositories

- Etc.



Integrates into CI/CD pipelines to manage infrastructure changes

Terraform is commonly used to define infrastructure as code.

Instead of manually creating cloud resources, teams define them in configuration files.

This ensures environments are reproducible and version-controlled.

Terraform can also be integrated directly into CI/CD pipelines to automatically provision infrastructure.

# Source management: Repositories

## Version control systems

- Store all code and configuration in version control
- GitHub, GitLab, Bitbucket

## Versioning is critical for agents

- Provides a single source of truth
- Enables collaboration and parallel development
- Creates an auditable history of all changes
- Allows for easy rollbacks to previous working versions

## Versioning prompts as artifacts

- Can also treat prompts as code: store them in version control
- Track changes to prompts to correlate with agent behavior changes

Version control systems such as GitHub or GitLab act as the source of truth for agent code.

For agent systems, it's also important to treat prompts as versioned artifacts.

Tracking prompt changes helps teams correlate behavior changes with updates.

Prompts = first-class artifacts

Versioning impacts behavior more than code

Version control also enables rollback to known working versions when issues arise.

# Artifact management: Model governance for agents

- Model strategy and selection
  - Use foundation models (e.g., Gemini family) for general capabilities
  - Use fine-tuned or adapter-based models for domain specialization
  - Maintain explicit mapping of agents → model versions → use cases
- Model registry and version control
  - Register all custom models in Model Registry
  - Enforce:
    - Semantic versioning (major/minor/patch)
    - Approval gates before production promotion
  - Track lineage: training data, hyperparameters, evaluation metrics

Models are another critical artifact in the pipeline.

Teams should maintain explicit mappings between agents, model versions, and use cases.

Agent Platform Model Registry can track model versions, metadata, and evaluation results.

## Artifact management: Model governance for agents

- Deployment and environment Isolation
  - Dev / Staging / Prod model versions
  - Canary deployments for agent rollout
  - Support rollback to prior stable model versions
  - Shadow traffic testing
  - Offline eval before rollout

Deployment strategies such as staging environments and canary releases help ensure safe model updates.

## Agent management: Agent2Agent (A2A) discovery strategies

### Well known URI

- A2A Server hosts their Agent Card at a well-known URI on their domain
- Best suited for open or domain-controlled discovery scenarios

### Curated Registries (Catalog-Based Discovery)

- Curated registry acts as a central repository
- A2A Servers publish their Agent Cards to the registry
- Clients can query and discover agents
- Current A2A specification does not prescribe a standard API for curated registries

### Direct Configuration / Private Discovery

- Clients are directly configured with Agent Card information or URLs
- Use for tightly coupled systems, private agents, or development purposes
- Inflexible for dynamic discovery scenarios

Agent-to-agent discovery mechanisms allow agents to locate and interact with other agents.

Several discovery strategies exist, including publishing agent cards at well-known URIs, using direct configuration, or registering agents in curated registries.

These approaches enable scalable multi-agent ecosystems.

## Artifact management: Tool catalog governance

- Tools are structured APIs/functions callable by agents
  - Define strict schemas (OpenAPI/JSON Schema)
  - Enforce input validation and output contracts
- Reliability and runtime controls
  - Monitor invocation latency and error rates
  - Define retry policies and circuit breakers
  - Require backward compatibility for minor versions
  - Provide safe fallback behavior on failure
- Security and access control
  - Enforce least-privilege access
  - Scope tools per agent or environment (dev/staging/prod)
  - Log all tool invocations with agent ID, model version, and user context

Agents often rely on tools or APIs.

Tool governance ensures tools are defined with strict schemas and reliability policies.

Operational controls such as retry policies, circuit breakers, and monitoring help maintain stability.

Security controls ensure that agents only access tools they are authorized to use.

# Central Tool Registry and MCP Control Plane

- MCP standardizes how agents discover and call tools, it defines:
  - Tool registration schema
  - Invocation format
  - Response structure
  - Optional streaming support
- MCP standardizes how agents:
  - Discover tools
  - Call tools
  - Receive structured results
- MCP publishing on Google Cloud
  - Apigee users with an existing API can publish their API as an MCP server
  - Developers who want to create their own MCP server can host it on Cloud Run

The Model Context Protocol standardizes how agents discover and invoke tools.

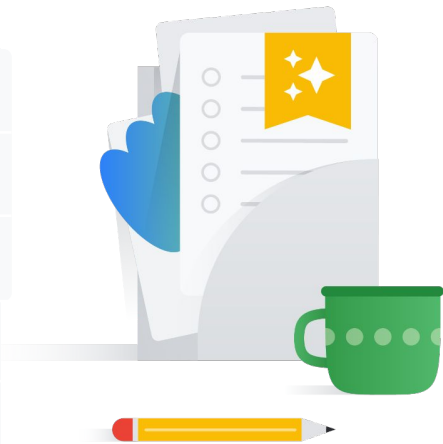
It defines schemas for tool registration, invocation formats, and structured responses.

On Google Cloud, APIs can be published as MCP servers using platforms such as Apigee or hosted services like Cloud Run.

This creates a consistent interface between agents and tools.

# Topics

- |    |                                            |
|----|--------------------------------------------|
| 01 | CI/CD review                               |
| 02 | Agentic deployment targets                 |
| 03 | CI/CD tooling and patterns on Google Cloud |
| 04 | <a href="#">Cloud Build automation</a>     |
| 05 | Lab: CI/CD for Agents on Google Cloud      |



Next, we focus on automating CI/CD pipelines using Cloud Build.

# Cloud Build automation

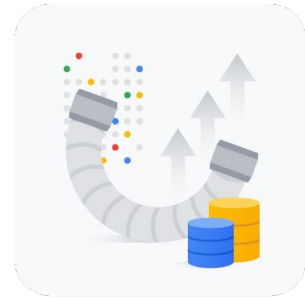
Pipeline steps are defined declaratively using a **cloudbuild.yaml** file

## Integrates with:

- GitHub
- GitHub Enterprise
- GitLab
- GitLab Enterprise
- Bitbucket Data Center
- Bitbucket Cloud

## Can trigger builds with:

- Push to a branch
- Push a new tag
- Pull request
- Manual invocation
- Pub/Sub message
- Webhook



Cloud Build integrates with several source repositories such as GitHub, GitLab, and Bitbucket.

Builds can be triggered by events such as pushing to a branch, creating a tag, submitting a pull request, or invoking the build manually.

This event-driven model allows pipelines to run automatically whenever code changes occur.

# Substituting build variable values

Use substitutions in `cloudbuild.yaml` to substitute specific variables at build time

Cloud Build provides default substitutions including:

- **\$PROJECT\_ID**: ID of your Cloud project
- **\$BUILD\_ID**: ID of your build
- **\$COMMIT\_SHA**: the commit ID of the build
- **\$BRANCH\_NAME**: the name of your branch
- **\$TAG\_NAME**: the name of your tag
- **\$PROJECT\_NUMBER**: your project number
- **\$REVISION\_ID**: the commit ID of the build

Can also create user-defined substitutions

- Define substitution variables on the trigger
- Must begin with an underscore

**Substitution variables**  
Substitutions allow re-use of a cloudbuild.yaml file with different variable values. Use bash string manipulation to combine variables and bindings to access arbitrary data in the JSON payload of the webhook. [Learn more](#)

|                                                        |                                                    |
|--------------------------------------------------------|----------------------------------------------------|
| Variable 1 *                                           | Value 1                                            |
| <input type="text" value="_APP_SERVICE_ACCOUNT_STA"/>  | <input type="text" value="demo-agent@steve-demo"/> |
| Variable 2 *                                           | Value 2                                            |
| <input type="text" value="_ARTIFACT_REGISTRY_REPO_N"/> | <input type="text" value="demo-agent-repo"/>       |

Cloud Build also supports substitution variables.

These allow pipeline configurations to dynamically insert values such as project IDs, commit hashes, or branch names.

Developers can also define custom substitution variables to parameterize pipelines and reuse configuration files.

## cloudbuild.yaml to deploy an agent to Cloud Run

```
steps:
  #Build Docker image
  - name: "gcr.io/cloud-builders/docker"
    args:
      [
        "build", "-t", "$_REGION-docker.pkg.dev/$PROJECT_ID/$_ARTIFACT_REGISTRY_REPO_NAME/$_CONTAINER_NAME",
        "--build-arg", "COMMIT_SHA=$COMMIT_SHA",
        ".",
      ]
  #Push Docker image to Artifact Registry
  - name: "gcr.io/cloud-builders/docker"
    args:
      [
        "push", "$_REGION-docker.pkg.dev/$PROJECT_ID/$_ARTIFACT_REGISTRY_REPO_NAME/$_CONTAINER_NAME",
      ]
  #Deploy Docker image to Cloud Run
  - name: "gcr.io/cloud-builders/gcloud"
    id: deploy-cloudrun
    entrypoint: gcloud
    args:
      - "run", "deploy", "lab-agent"
      - "--image", "$_REGION-docker.pkg.dev/$PROJECT_ID/$_ARTIFACT_REGISTRY_REPO_NAME/$_CONTAINER_NAME"
      - "--region", "${_REGION}"
      - "--project", "${_PROJECT_ID}"
```

This example shows a Cloud Build pipeline used to deploy an agent to Cloud Run.

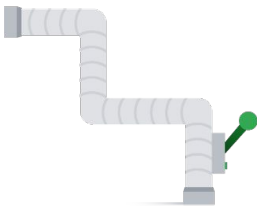
The pipeline performs three main steps:

- First, it builds a Docker container image.
- Second, it pushes that image to Artifact Registry.
- Finally, it deploys the container to Cloud Run.

This entire process can be triggered automatically whenever code changes are committed.

# Cloud Build worker pools

- By default, each build runs on a worker from a hosted default pool
- Can optionally create private pools
- Both pools are:
  - Fully managed
  - Scale to zero



## Default pools

Each build runs on a hosted worker and is isolated from other workloads

## Private Pools

Private, dedicated pools of workers that offer greater customization

Cloud Build executes builds using worker pools.

By default, builds run on hosted workers provided by Google.

Organizations can also create private worker pools to gain more control over networking, performance, and security.

Private pools can run inside VPC environments and provide greater customization options.

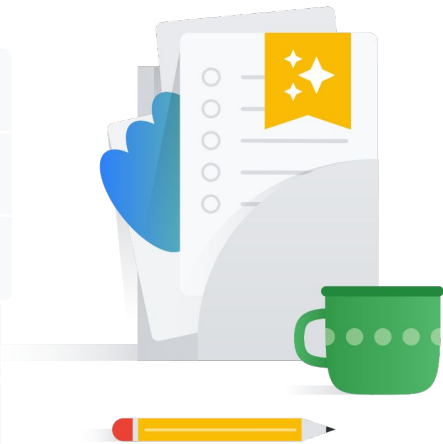
## Cloud Build worker pool features

| Feature                                                   | Default pool                                                                      | Private pool                                                                        |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Access to public internet                                 |  | Configurable                                                                        |
| Peer into a VPC or shared VPC to access private resources |  |  |
| Static internal IP ranges                                 |  |  |
| Ability to disable public IPs                             |  |  |
| Maximum concurrent builds                                 | 30                                                                                | 100+                                                                                |
| Machine types                                             | 5                                                                                 | 64                                                                                  |

This table lists some of the features of the default pool compared to Private Pools. Private pools provide more control and flexibility.

# Topics

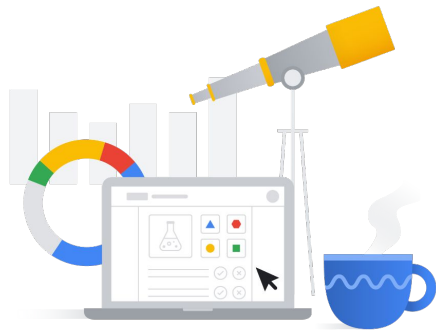
- |    |                                                       |
|----|-------------------------------------------------------|
| 01 | CI/CD review                                          |
| 02 | Agentic deployment targets                            |
| 03 | CI/CD tooling and patterns on Google Cloud            |
| 04 | Cloud Build automation                                |
| 05 | <a href="#">Lab: CI/CD for Agents on Google Cloud</a> |



Now that we've covered the theory and tooling behind CI/CD pipelines, we'll move into a hands-on lab.

This lab will demonstrate how these concepts come together in a real deployment workflow.

# Lab: Building CI/CD Pipelines for Agents with Terraform and Cloud Build



Google Cloud

01 Set up GitHub and test an agent locally

02 Configure terraform to create the CI/CD pipeline

03 Trigger the CI/CD pipeline to build and deploy the agent to Cloud Run

04 Modify the code and verify it deploys automatically

05 Adding tests to the pipeline

Proprietary & Confidential 33

In this lab, you:

- Set up a GitHub repository and test an agent locally.
- Use Terraform to configure the CI/CD infrastructure.
- Trigger the pipeline to build and deploy the agent to Cloud Run.
- Modify the code to verify that the pipeline redeploys automatically.

## Questions and answers



You've now learned how CI/CD pipelines support agent deployments and how Google Cloud services can automate that process.

Any questions before we continue?



Thank you.  
Module 2 is complete.